Aim: A* Search algorithm for solving a pathfinding problem.

```
import heapq

import matplotlib.pyplot as plt

import networkx as nx

import textwrap



# Carol Pillai T104

graph = {

    'MVLU College':               {'Western Exp Highway': 1.00},

    'Western Exp Highway':        {'Mumbai Airport': 2.20},

    'Mumbai Airport':             {'Kalanagar Flyover': 5.00},

    'Kalanagar Flyover':          {'Bandra Reclamation Flyover': 1.60},

    'Bandra Reclamation Flyover': {'Basilica of Our Lady of the Mount':
1.41},

    'Basilica of Our Lady of the Mount': {}

}



Carol Pillai
T104
```

```
heuristics = {

    'MVLU College': 11.21,

    'Western Exp Highway': 10.21,

    'Mumbai Airport': 8.01,

    'Kalanagar Flyover': 3.01,

    'Bandra Reclamation Flyover': 1.41,

    'Basilica of Our Lady of the Mount': 0

}



node_positions = {

    'MVLU College':                     (2, 10),

    'Western Exp Highway':              (2.1, 8),

    'Mumbai Airport':                   (2.0, 6),

    'Kalanagar Flyover':                (1.9, 4),

    'Bandra Reclamation Flyover':       (1.7, 2),

    'Basilica of Our Lady of the Mount':  (1.5, 0)
```

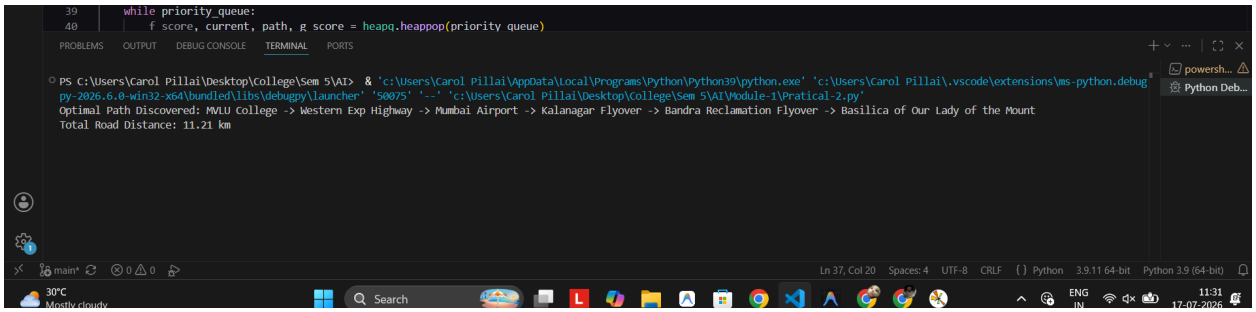
```
}
```

```
def a_star_search(graph, heuristics, start, goal):

    priority_queue = [(heuristics[start], start, [start], 0)]

    visited = set()



    while priority_queue:

        f_score, current, path, g_score = heapq.heappop(priority_queue)

        if current in visited:

            continue

        visited.add(current)

        if current == goal:

            return path, g_score

        for neighbor, edge_weight in graph[current].items():

            if neighbor not in visited:
```

```
                next_g = g_score + edge_weight

                next_f = next_g + heuristics[neighbor]

                heapq.heappush(priority_queue, (next_f, neighbor, path +
[neighbor], next_g))




    return None, float('inf')



optimal_path, total_distance = a_star_search(graph, heuristics, 'MVLU
College', 'Basilica of Our Lady of the Mount')

print(f"Optimal Path Discovered: {' -> '.join(optimal_path)}")

print(f"Total Road Distance: {round(total_distance, 2)} km\n")







G = nx.DiGraph()

for node, neighbors in graph.items():

    for neighbor, weight in neighbors.items():

        G.add_edge(node, neighbor, weight=weight)
```

```
plt.figure(figsize=(10, 13))



path_edges = list(zip(optimal_path, optimal_path[1:]))

normal_edges = [edge for edge in G.edges() if edge not in path_edges]



nx.draw_networkx_nodes(G, node_positions, node_size=7000,
node_color='lightblue')

nx.draw_networkx_edges(G, node_positions, edgelist=normal_edges,
width=1.5, edge_color='gray', arrows=True,

                       min_source_margin=40, min_target_margin=40)

nx.draw_networkx_edges(G, node_positions, edgelist=path_edges, width=3.5,
edge_color='darkorange', arrows=True,

                       min_source_margin=40, min_target_margin=40)



node_labels = {

    node: f"{textwrap.fill(node, 12)}\nh(n)={heuristics[node]}"

    for node in G.nodes()
```

```
}

nx.draw_networkx_labels(G, node_positions, labels=node_labels,
font_size=9, font_weight='bold')




edge_labels = nx.get_edge_attributes(G, 'weight')

formatted_edge_labels = {k: f"{v} km" for k, v in edge_labels.items()}

nx.draw_networkx_edge_labels(G, node_positions,
edge_labels=formatted_edge_labels, font_color='red', font_size=10)




plt.title("A* Routing Map: MVLU College to Basilica of Our Lady of the
Mount\n(Highlighted Orange Path = Optimal Route)", fontsize=12)

plt.axis('off')

plt.tight_layout()

plt.show()
```


while priority_queue:
f score, current, path, g score = heapq.heappop(priority queue)
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\Carol Pillai\Desktop\College\Sem 5\AI> & 'c:\Users\Carol Pillai\AppData\Local\Programs\Python\Python39\python.exe' 'c:\Users\Carol Pillai\.vscode\extensions\ms-python.debug
py-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '50075' '--' 'c:\Users\Carol Pillai\Desktop\College\Sem 5\AI\Module-1\Pratical-2.py'
Optimal Path Discovered: MVLU College -> Western Exp Highway -> Mumbai Airport -> Kalanagar Flyover -> Bandra Reclamation Flyover -> Basilica of Our Lady of the Mount
Total Road Distance: 11.21 km
Python Deb...
Ln 37, Col 20
Spaces: 4
UTF-8
CRLF
Python
3.9.11 64-bit
Python 3.9 (64-bit)
30°C
Mostly cloudy
Search
ENG
IN
11:31
17-07-2026

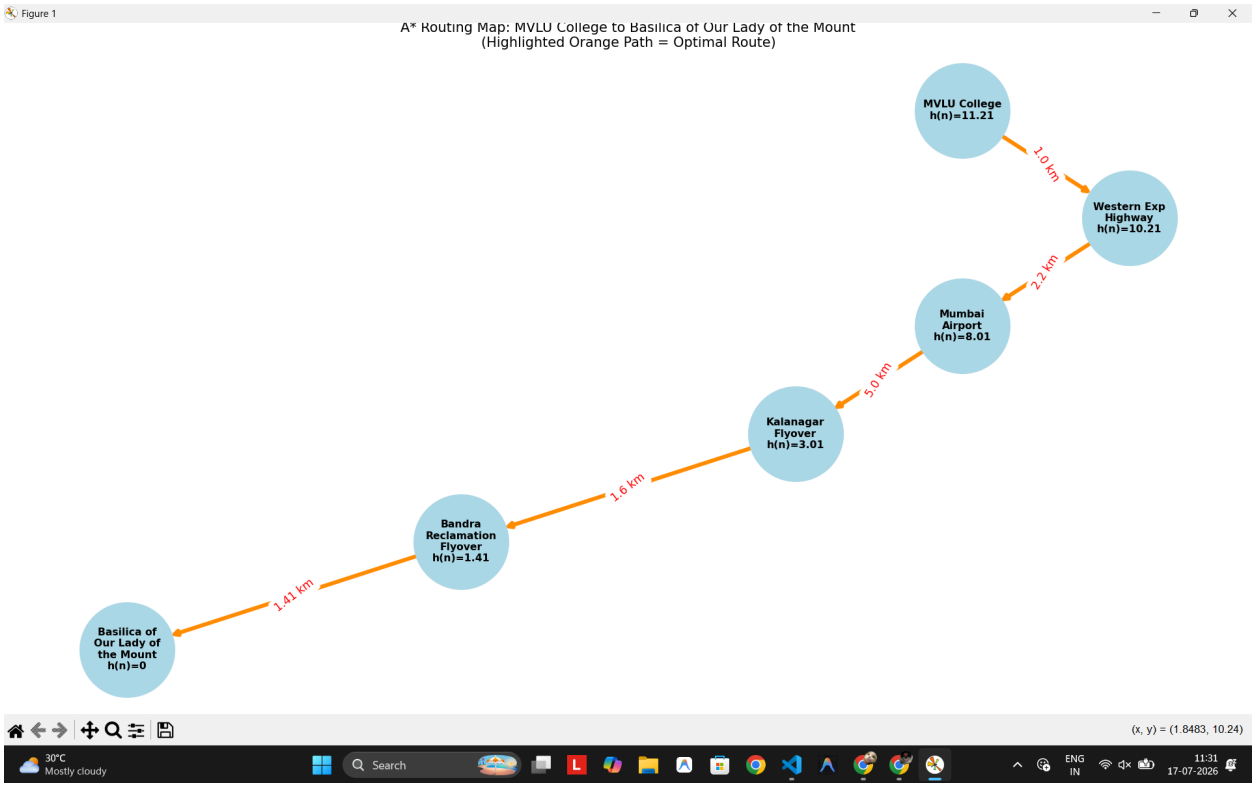


Figure 1
A* Routing Map: MVLU College to Basilica of Our Lady of the Mount
(Highlighted Orange Path = Optimal Route)
MVLU College
h(n)=11.21
1.0 km
Western Exp
Highway
h(n)=10.21
2.2 km
Mumbai
Airport
h(n)=8.01
5.0 km
Kalanagar
Flyover
h(n)=3.01
1.6 km
Bandra
Reclamation
Flyover
h(n)=1.41
1.41 km
Basilica of
Our Lady of
the Mount
h(n)=0
(x, y) = (1.8483, 10.24)
30°C
Mostly cloudy
Search
ENG
IN
11:31
17-07-2026

Recursive Best-First Search Algorithm (RBFS)

```
import matplotlib.pyplot as plt




# Carol Pillai T104

graph = {

    'MVLU College':               {'Western Exp Highway': 1.00},

    'Western Exp Highway':        {'Mumbai Airport': 2.20},

    'Mumbai Airport':             {'Kalanagar Flyover': 5.00},

    'Kalanagar Flyover':          {'Bandra Reclamation Flyover': 1.60},

    'Bandra Reclamation Flyover': {'Basilica of Our Lady of the Mount':
1.41},

    'Basilica of Our Lady of the Mount': {}

}




heuristics = {

    'MVLU College': 11.21,

    'Western Exp Highway': 10.21,
```

```
    'Mumbai Airport': 8.01,

    'Kalanagar Flyover': 3.01,

    'Bandra Reclamation Flyover': 1.41,

    'Basilica of Our Lady of the Mount': 0

}



coords = {

    'MVLU College': (2, 14),

    'Western Exp Highway': (2.1, 11.2),

    'Mumbai Airport': (2.0, 8.4),

    'Kalanagar Flyover': (1.9, 5.6),

    'Bandra Reclamation Flyover': (1.7, 2.8),

    'Basilica of Our Lady of the Mount': (1.5, 0)

}
```

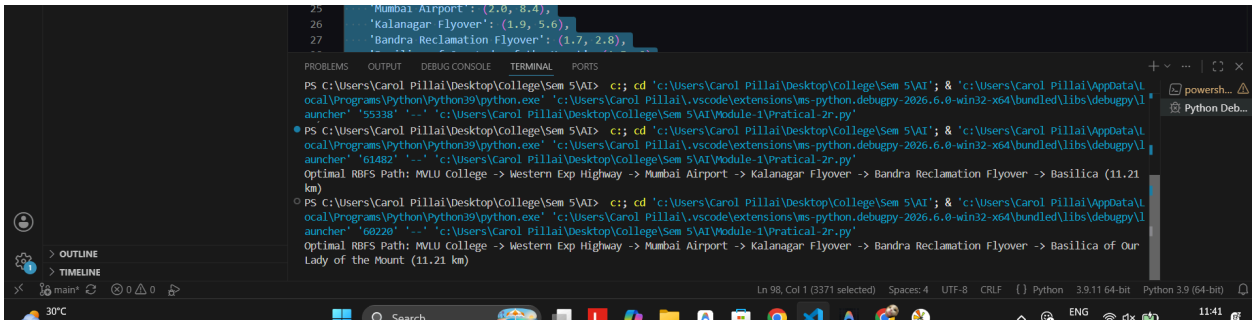
```
def rbfs_search(start, goal):

    success, path, cost, _ = rbfs(start, goal, g=0, f_limit=float('inf'),
path=[start])

    return path, cost




def rbfs(node, goal, g, f_limit, path):

    if node == goal:

        return True, path, g, g



    neighbors = graph[node]

    if not neighbors:

        return False, [], 0, float('inf')



    successors = []

    for neighbor, distance in neighbors.items():

        if neighbor not in path:

            next_g = g + distance
```

```
            next_f = max(next_g + heuristics[neighbor], g +
heuristics[node])

            successors.append([next_f, neighbor, next_g])



    if not successors:

        return False, [], 0, float('inf')



    while True:

        successors.sort(key=lambda x: x[0])

        best = successors[0]



        if best[0] > f_limit:

            return False, [], 0, best[0]



        alternative_f = successors[1][0] if len(successors) > 1 else
float('inf')
```

```
        success, result_path, total_g, returned_f = rbfs(

            best[1], goal, best[2], min(f_limit, alternative_f), path +
[best[1]]

        )



        best[0] = returned_f



        if success:

            return True, result_path, total_g, returned_f



# Run the algorithm

path, total_dist = rbfs_search('MVLU College', 'Basilica of Our Lady of
the Mount')

print(f"Optimal RBFS Path: {' -> '.join(path)} ({round(total_dist, 2)}
km)")







plt.figure(figsize=(11, 14))
```

```
for node, neighbors in graph.items():

    x1, y1 = coords[node]

    for neighbor, dist in neighbors.items():

        x2, y2 = coords[neighbor]

        is_path = node in path and neighbor in path and
path.index(neighbor) == path.index(node) + 1

        color, width = ('#2ecc71', 3) if is_path else ('#bdc3c7', 1.5)

        plt.plot([x1, x2], [y1, y2], color=color, linewidth=width,
zorder=1)

        plt.text((x1 + x2) / 2, (y1 + y2) / 2, f"{dist}km", color='red',
fontsize=10, ha='center')



for node, (x, y) in coords.items():

    color = '#f1c40f' if node in path else '#3498db'

    plt.scatter(x, y, color=color, s=9000, zorder=2, edgecolors='black',
linewidths=1)

    plt.text(x, y, f"{node}\nh={heuristics[node]}", ha='center',
va='center',
```

```
                color='black', fontsize=8, fontweight='bold', wrap=True)




plt.title(f"RBFS Shortest Route Map (Total: {round(total_dist, 2)}km)",
fontsize=13, fontweight='bold')

plt.axis('off')

plt.tight_layout()

plt.show()
```


'Mumbai Airport': (2.0, 8.4),
'Kalanagar Flyover': (1.9, 5.6),
'Bandra Reclamation Flyover': (1.7, 2.8),
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Carol Pillai\Desktop\College\Sem 5\AI> c:; cd 'c:\Users\Carol Pillai\Desktop\College\Sem 5\AI'; & 'c:\Users\Carol Pillai\AppData\Local\Programs\Python\Python39\python.exe' 'c:\Users\Carol Pillai\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '55338' '--' 'c:\Users\Carol Pillai\Desktop\College\Sem 5\AI\Module-1\Pratical-2r.py'
PS C:\Users\Carol Pillai\Desktop\College\Sem 5\AI> c:; cd 'c:\Users\Carol Pillai\Desktop\College\Sem 5\AI'; & 'c:\Users\Carol Pillai\AppData\Local\Programs\Python\Python39\python.exe' 'c:\Users\Carol Pillai\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '61482' '--' 'c:\Users\Carol Pillai\Desktop\College\Sem 5\AI\Module-1\Pratical-2r.py'
Optimal RBFS Path: MVLU College -> Western Exp Highway -> Mumbai Airport -> Kalanagar Flyover -> Bandra Reclamation Flyover -> Basilica (11.21 km)
PS C:\Users\Carol Pillai\Desktop\College\Sem 5\AI> c:; cd 'c:\Users\Carol Pillai\Desktop\College\Sem 5\AI'; & 'c:\Users\Carol Pillai\AppData\Local\Programs\Python\Python39\python.exe' 'c:\Users\Carol Pillai\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '60220' '--' 'c:\Users\Carol Pillai\Desktop\College\Sem 5\AI\Module-1\Pratical-2r.py'
Optimal RBFS Path: MVLU College -> Western Exp Highway -> Mumbai Airport -> Kalanagar Flyover -> Bandra Reclamation Flyover -> Basilica of Our Lady of the Mount (11.21 km)
OUTLINE
TIMELINE
Ln 98, Col 1 (3371 selected) Spaces: 4 UTF-8 CRLF Python 3.9.11 64-bit Python 3.9 (64-bit)

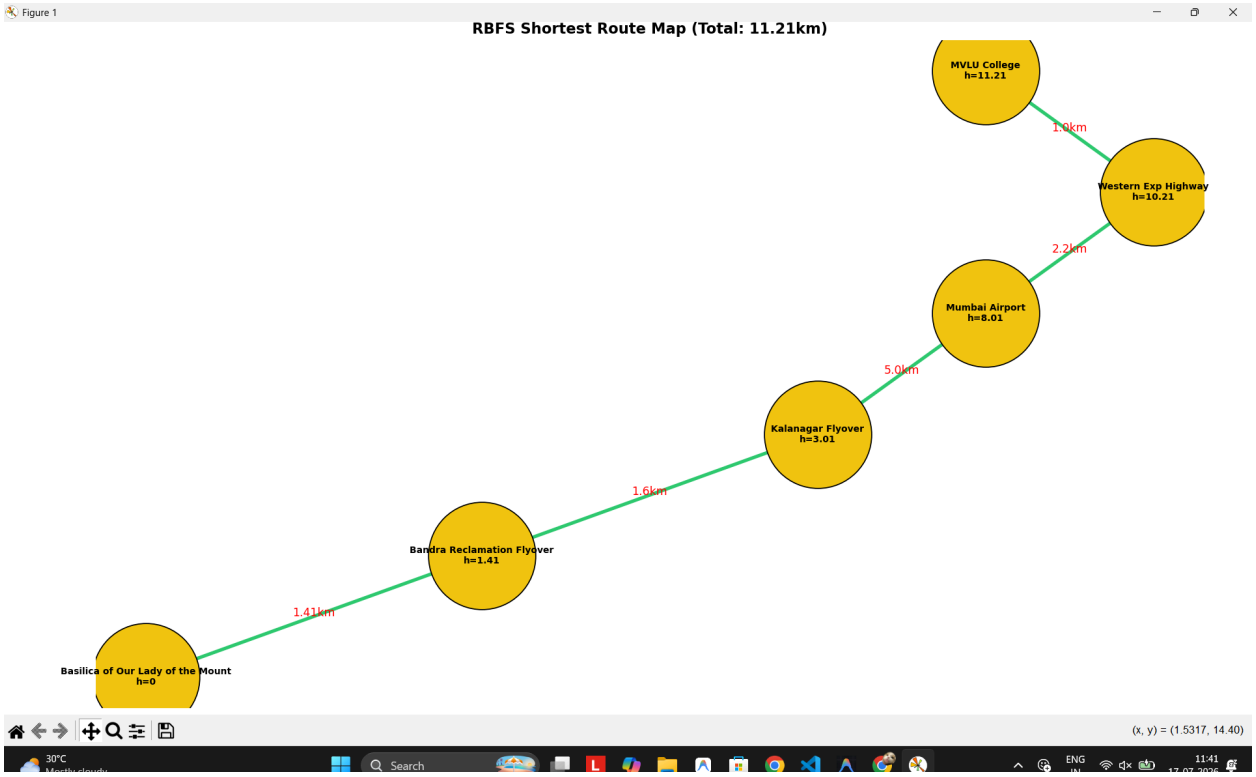


Figure 1
RBFS Shortest Route Map (Total: 11.21km)
MVLU College
h=11.21
1.0km
Western Exp Highway
h=10.21
2.2km
Mumbai Airport
h=8.01
5.0km
Kalanagar Flyover
h=3.01
1.6km
Bandra Reclamation Flyover
h=1.41
1.41km
Basilica of Our Lady of the Mount
h=0
(x, y) = (1.5317, 14.40)